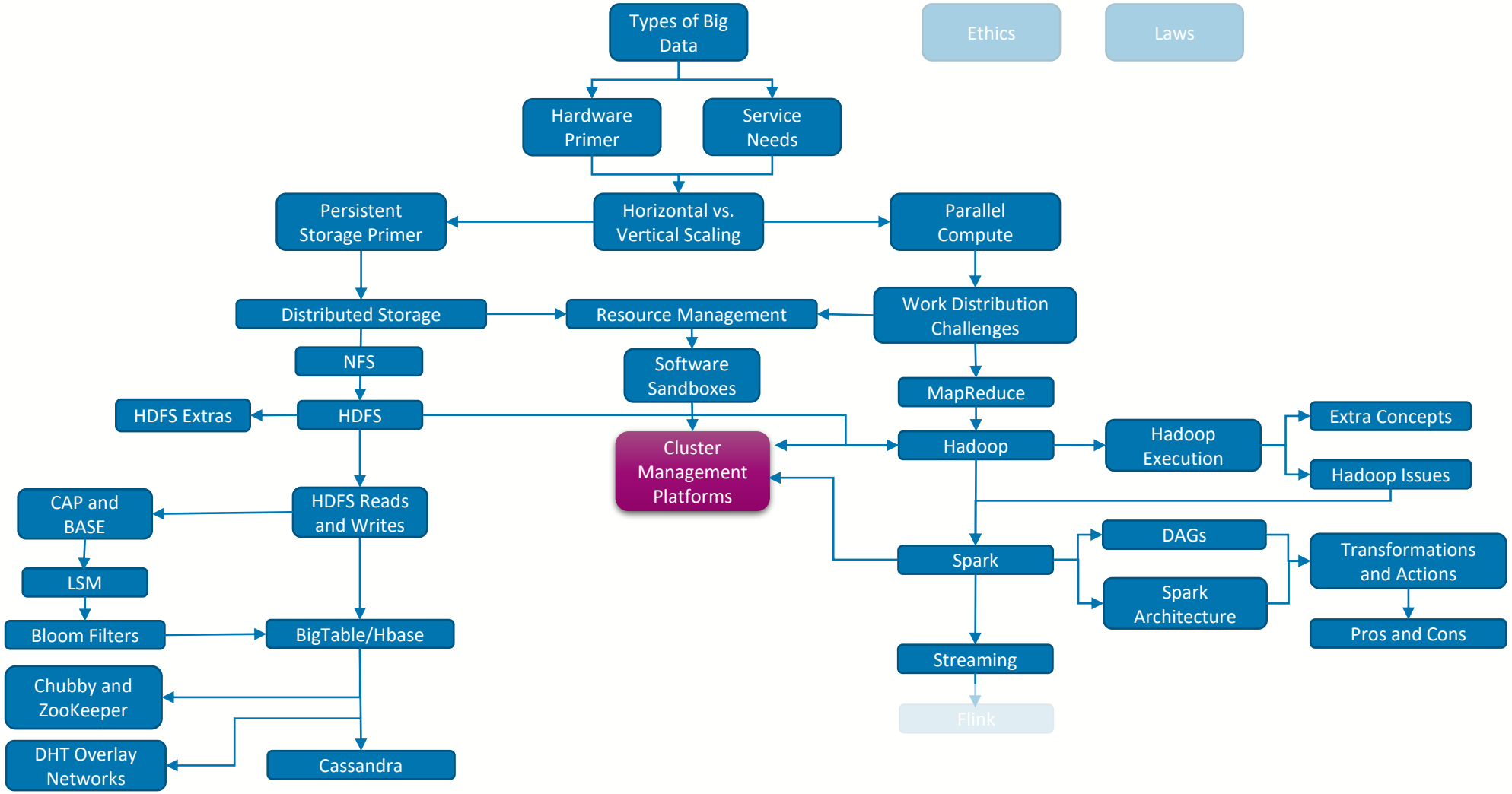
W10-01: Cluster Management Platforms

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

University of Glasgow

Where are we?



University of Glasgow

Cluster Management

- So, we now know that we can use sandboxing (of some type or another) to allow us to package and then run a service or application on a (remote) machine without running into incompatibilities

- But if we are running a large cluster, we don't want to have to setup those sandboxes and manage them manually

- This is the role of a **Cluster/Resource Management Platform**
  - Enable sandboxes to be automatically deployed to physical machines in the cluster on-demand
  - Track and manage available resources provided on the cluster, such that deployment of services intelligently accounts for where resources are available

University of Glasgow

University of Glasgow

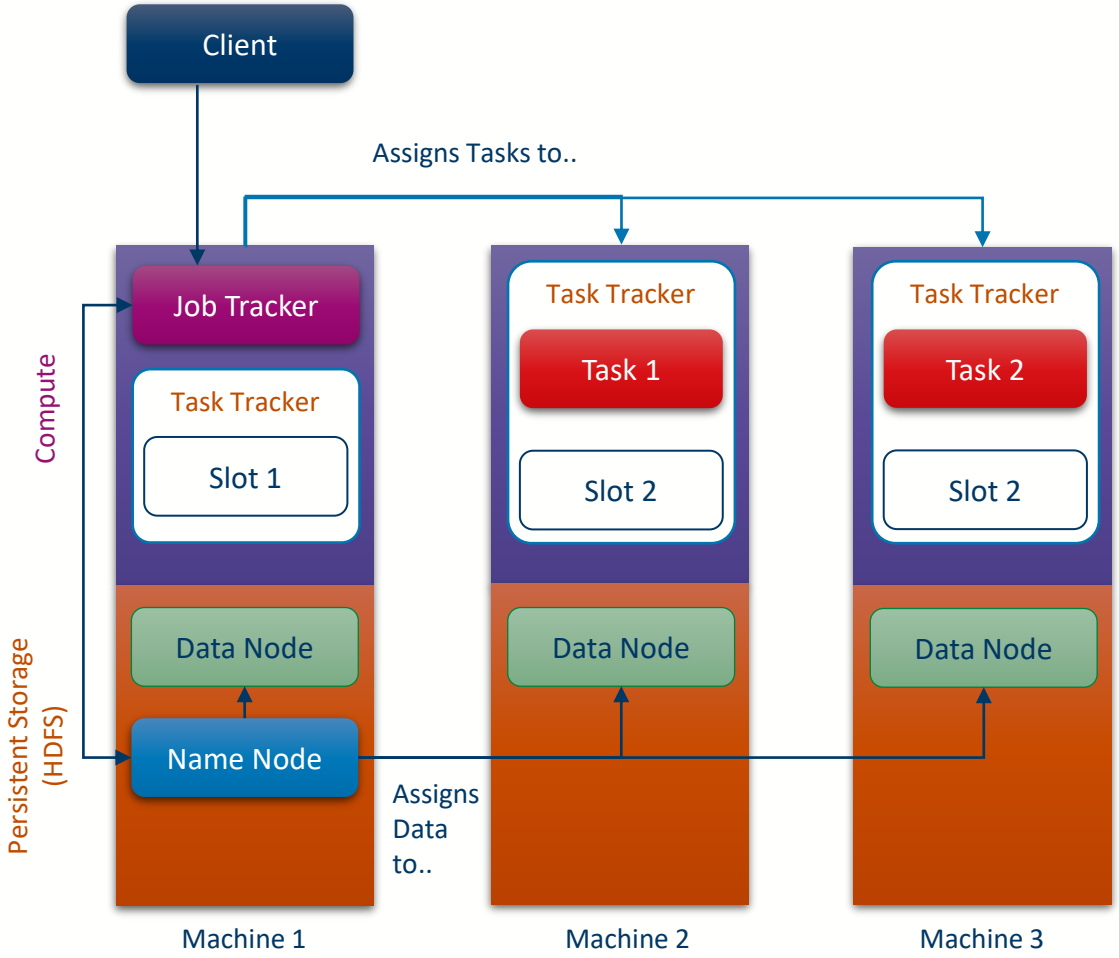
The Hadoop Solution: YARN

- One of the earlier solutions to this problem is Apache YARN (Yet Another Resource Negotiator)
  - YARN is a scheduling system designed for use with applications within the Hadoop ecosystem (although you can integrate other applications)

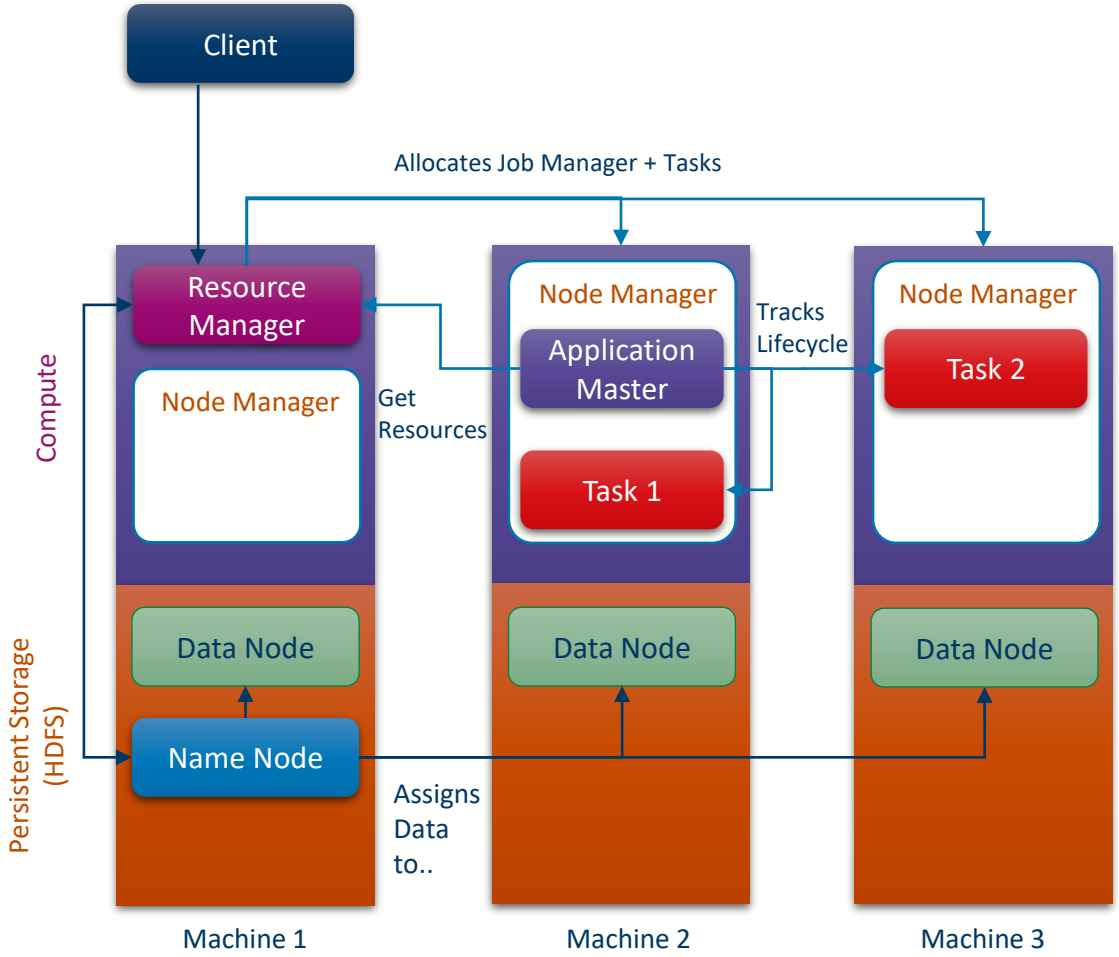
- Started from the observation that the Job Tracker in Hadoop has a lot of responsibilities
  - Scheduling tasks belonging to a job
  - Monitoring task executions
  - Restart failed tasks and run speculative tasks
  - Calculate counters (Hadoop Accumulators)

- Goal: Separate Cluster Management from Job Coordination
  - Use many workers to manage the Job lifecycles instead

The Hadoop Solution: YARN



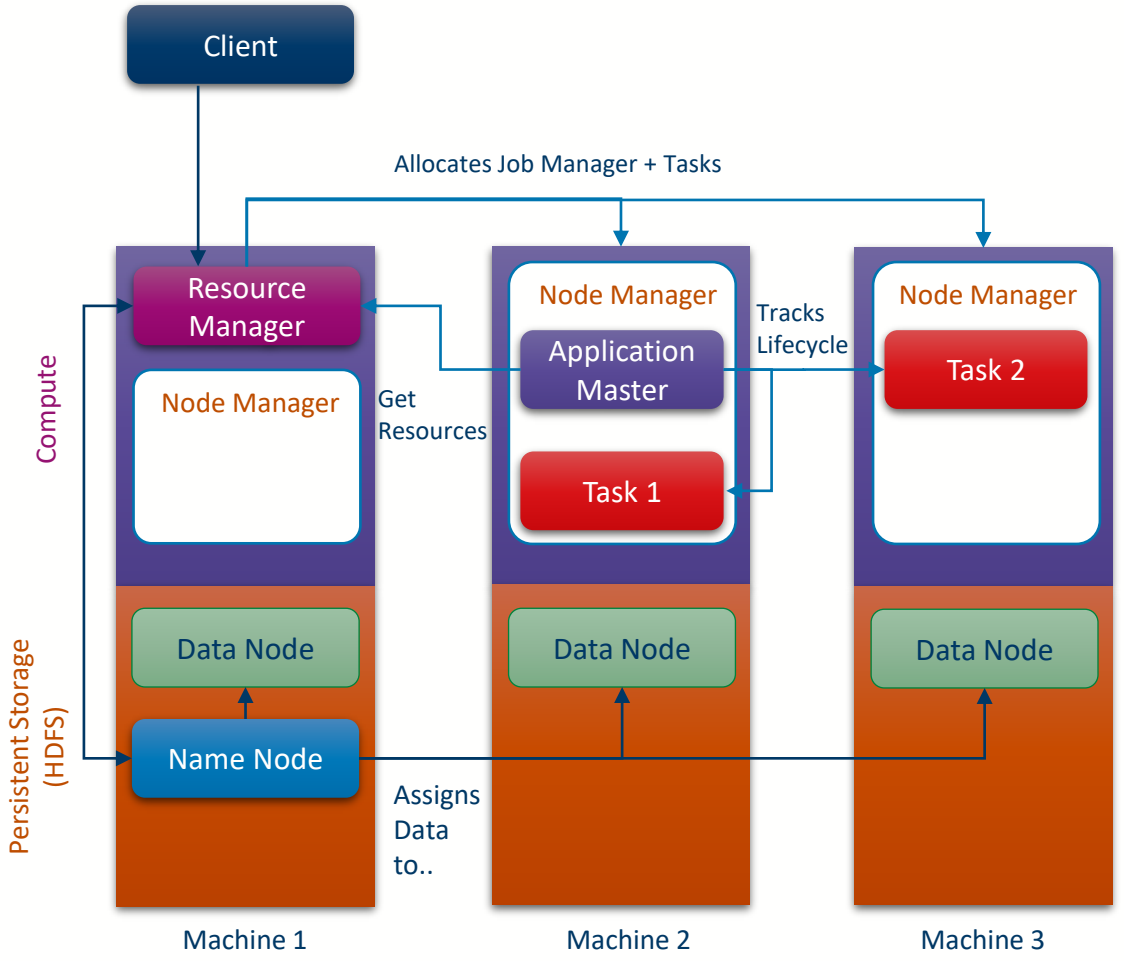
"Classical" Hadoop



YARN

The Hadoop Solution: YARN

- Differences
  - New Resource Manager – is responsible for managing the Hadoop Cluster
    - Assigns Application Managers and Tasks to machines
  - New Node Manager – replaces the task tracker
    - More flexible than the task tracker – can run any compatible task
    - Tracks actual machine resources and can support different 'sized' tasks
    - Defines blocks of resources for tasks, known as NodeManager Containers
      - Not to be confused with normal containers
  - New Application Master replaces the Job Tracker
    - Only tracks the job/application lifecycle
    - Asks the resource manager where tasks should go
    - More lightweight
    - Takes a task tracker slot
    - More than just MapReduce jobs

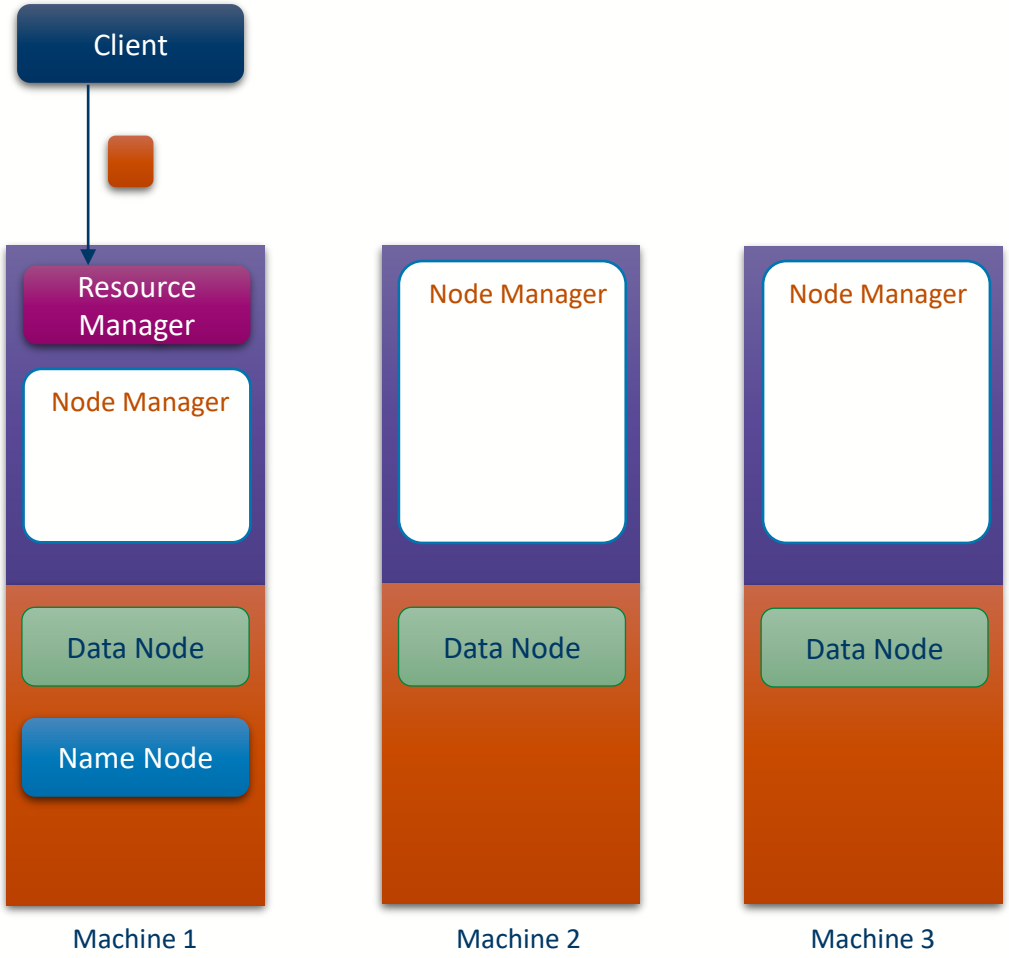


YARN

University of Glasgow

YARN Application Lifecycle

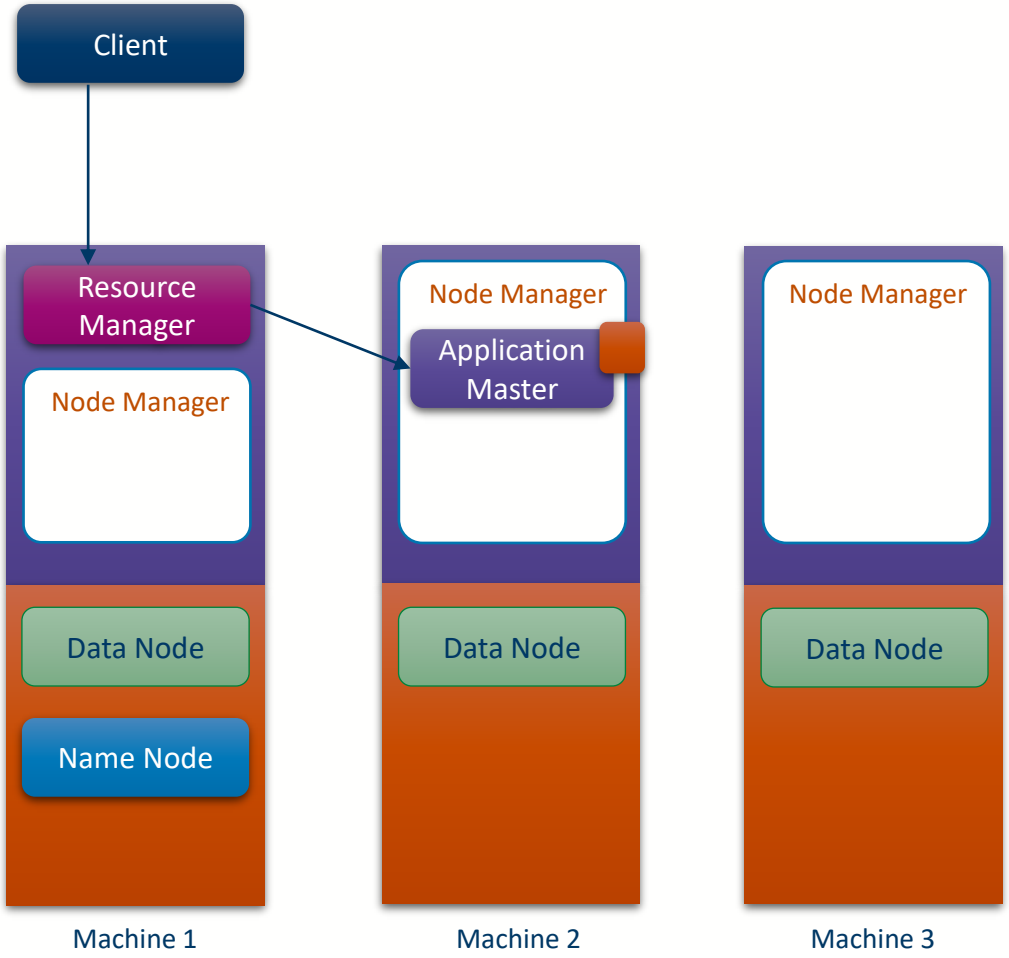
- Client registers a new application with the resource manager



University of Glasgow

YARN Application Lifecycle

- Resource Manager starts an Application Master to manage the application

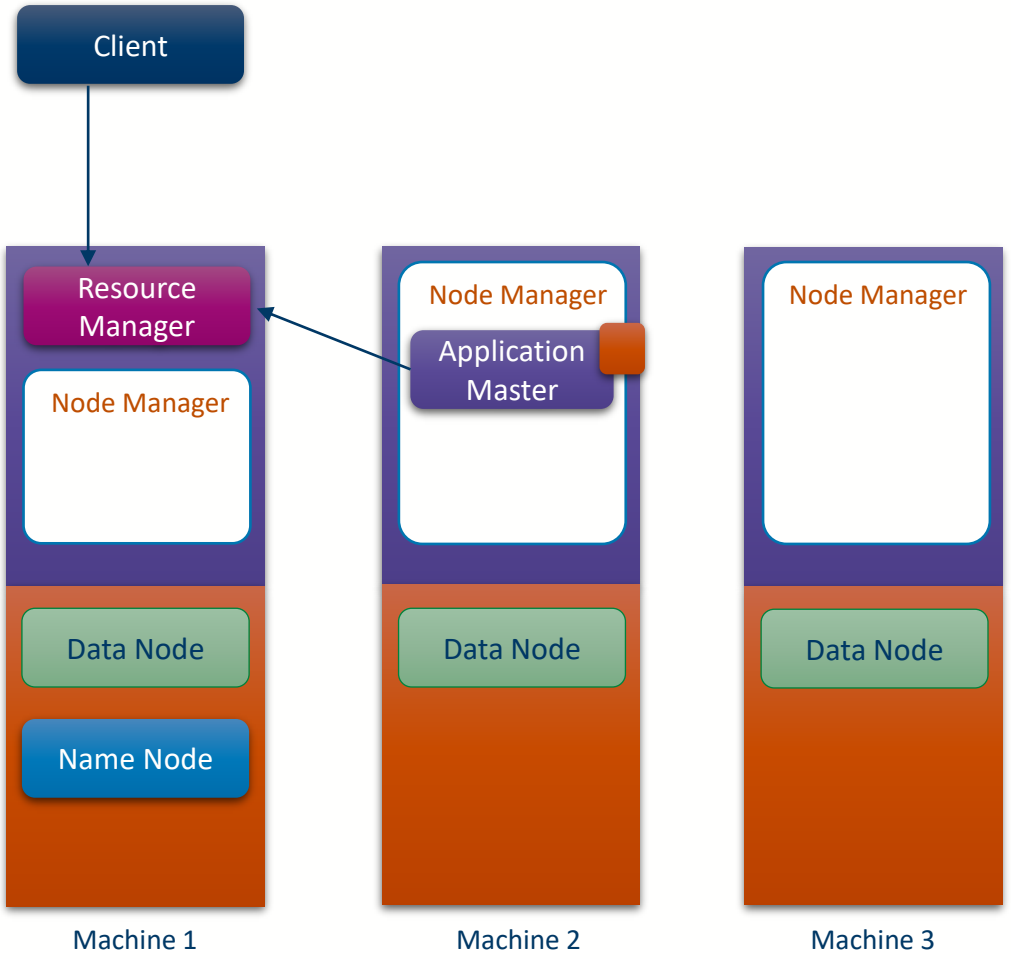


University of Glasgow

YARN Application Lifecycle

- The new application master asks for resources to run the application from the resource manager

- The resource manager returns with a list of containers to start and where
  - Priority
  - Hostname
  - Resources
  - Number of Containers

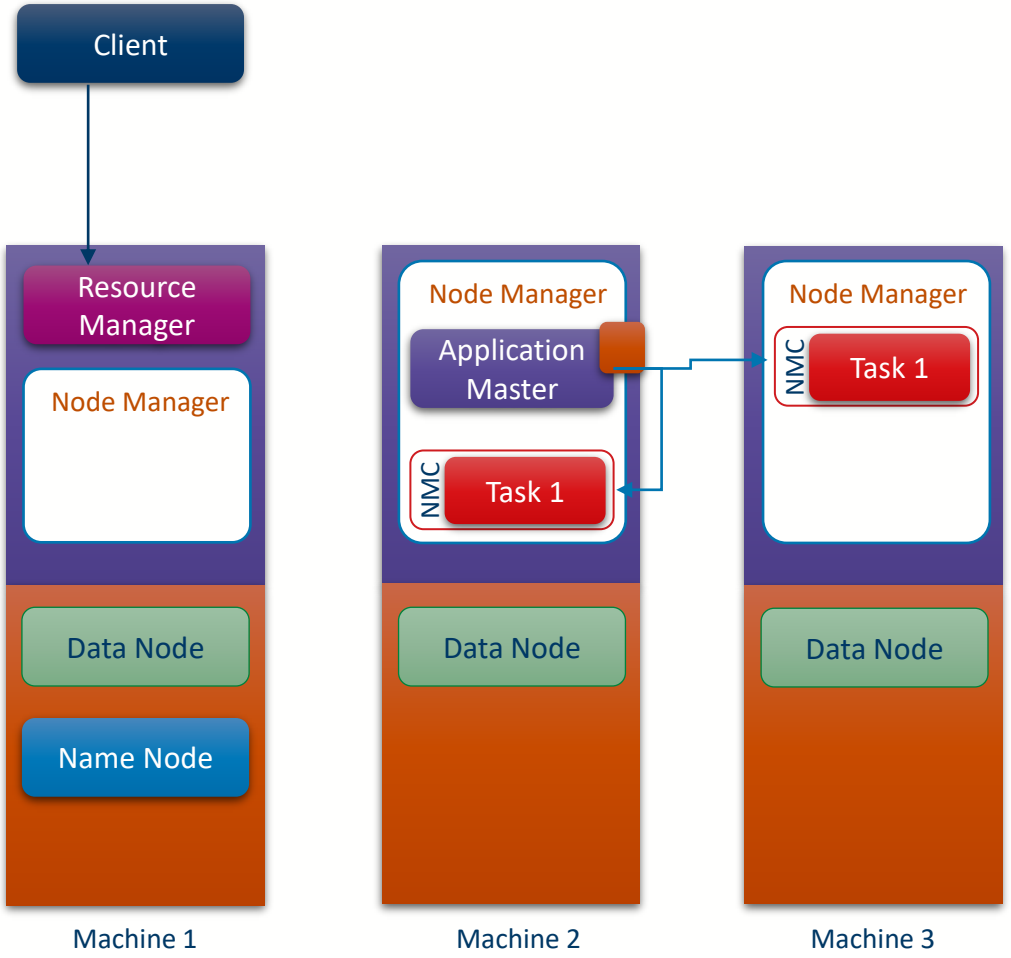


University of Glasgow

YARN Application Lifecycle

- The application master asks the associated Node Managers to start containers for each task with the specified resources

- From this point the logic will depend on the type of the application
  - Managed by the application master



University of Glasgow

Resource and Node Managers

Resource Manager

- Comprised of two services
  - Scheduler: Just responsible for examining resources and allocating tasks/application masters to node managers
  - Application Manager: Responsible for new application registration, first application master boot and any needed restarts

Node Manager

- Responsible for allocating containers to run tasks
- Also contains shuffle handlers to enable data passing between tasks (e.g. from Map to Reduce)
- Does not support JVM reuse for tasks within an application

University of Glasgow

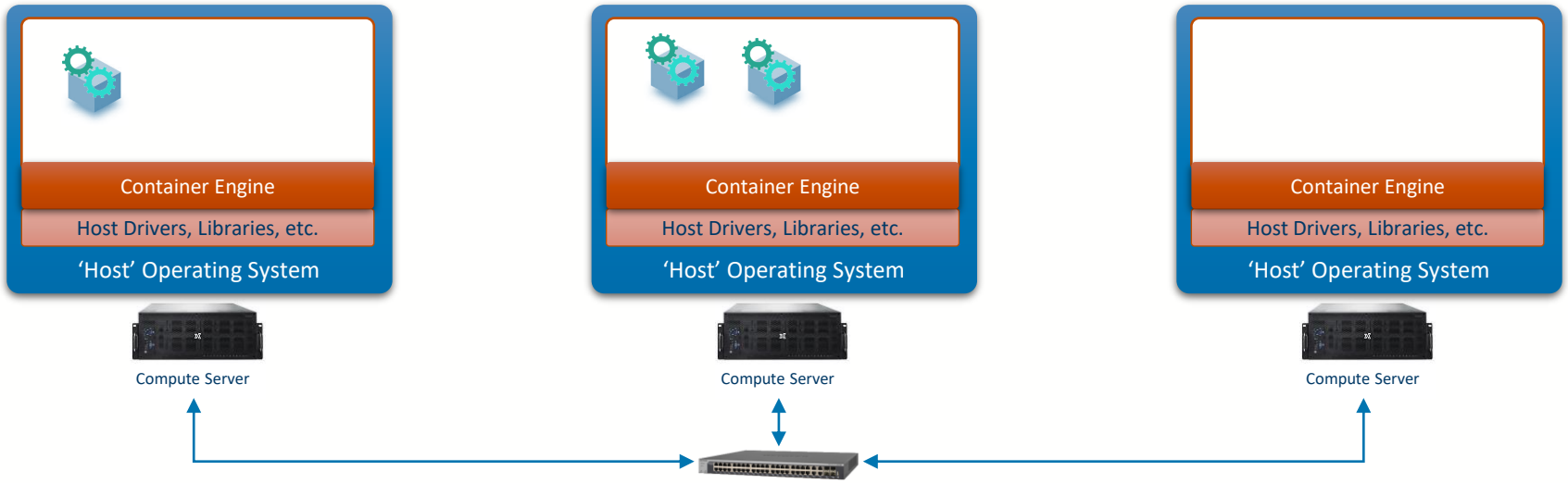
kubernetes

University of Glasgow

Container Management

- YARN is a reasonable solution if you are interested in managing JVM-based processes in a distributed fashion (e.g. Spark Clusters)

- However, with the advent of Container formats and engines like Docker, a different management system is needed
  - How can we coordinate different container engines?



University of Glasgow

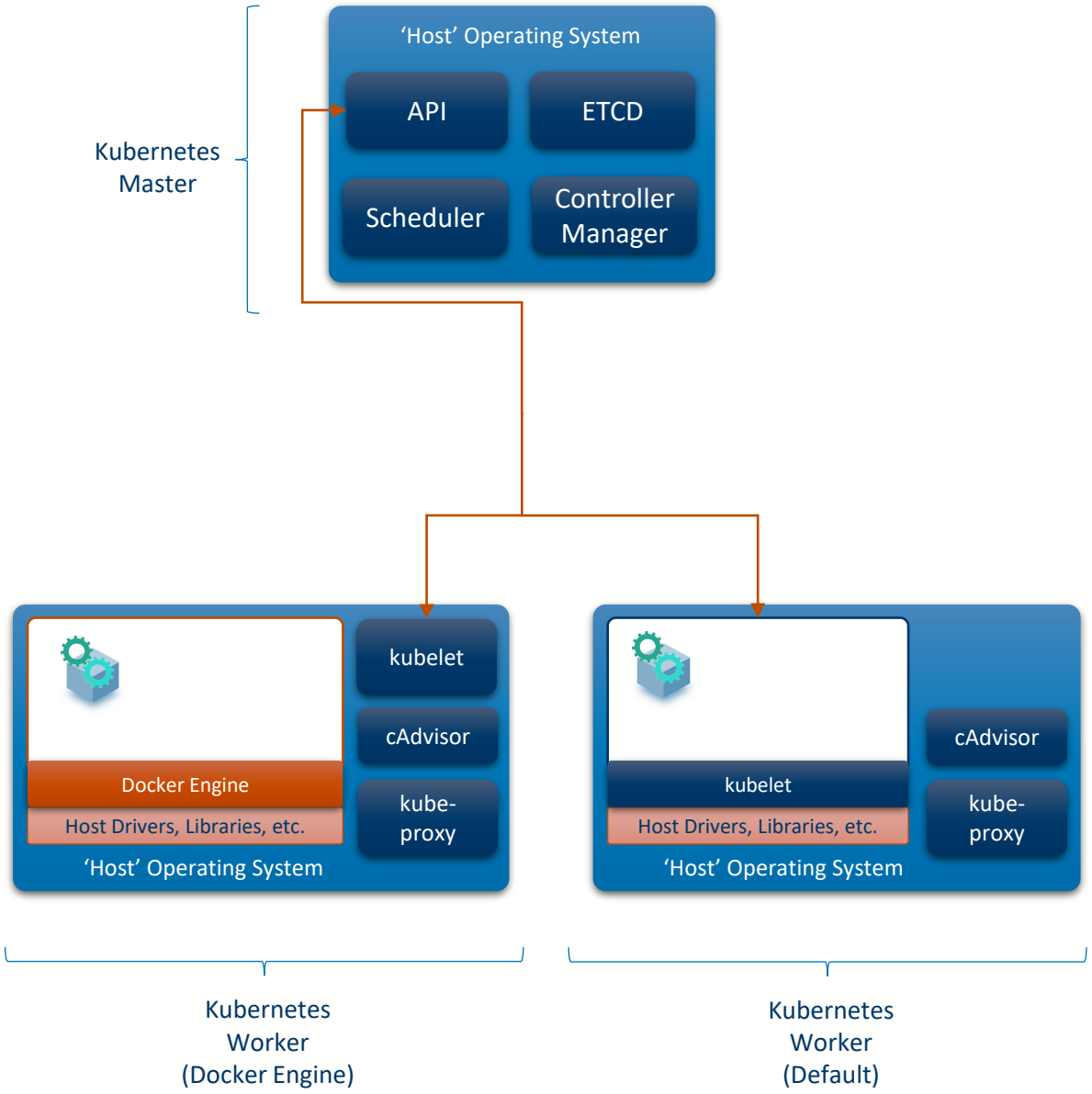
Kubernetes

- Kubernetes is an open-source container orchestration system for automating software deployment, scaling, and management.
  - Originally developed by Google, now maintained by the Cloud Native Computing Foundation

- Kubernetes provides
  - An API that allows for new containers and associated configuration to be registered
  - A central database (ETCD) that holds information about containers, configuration and state
  - A scheduler that allocates containers to container engines
  - A controller manager that runs various maintenance processes (e.g. failure recovery, node status tracking and container replication)

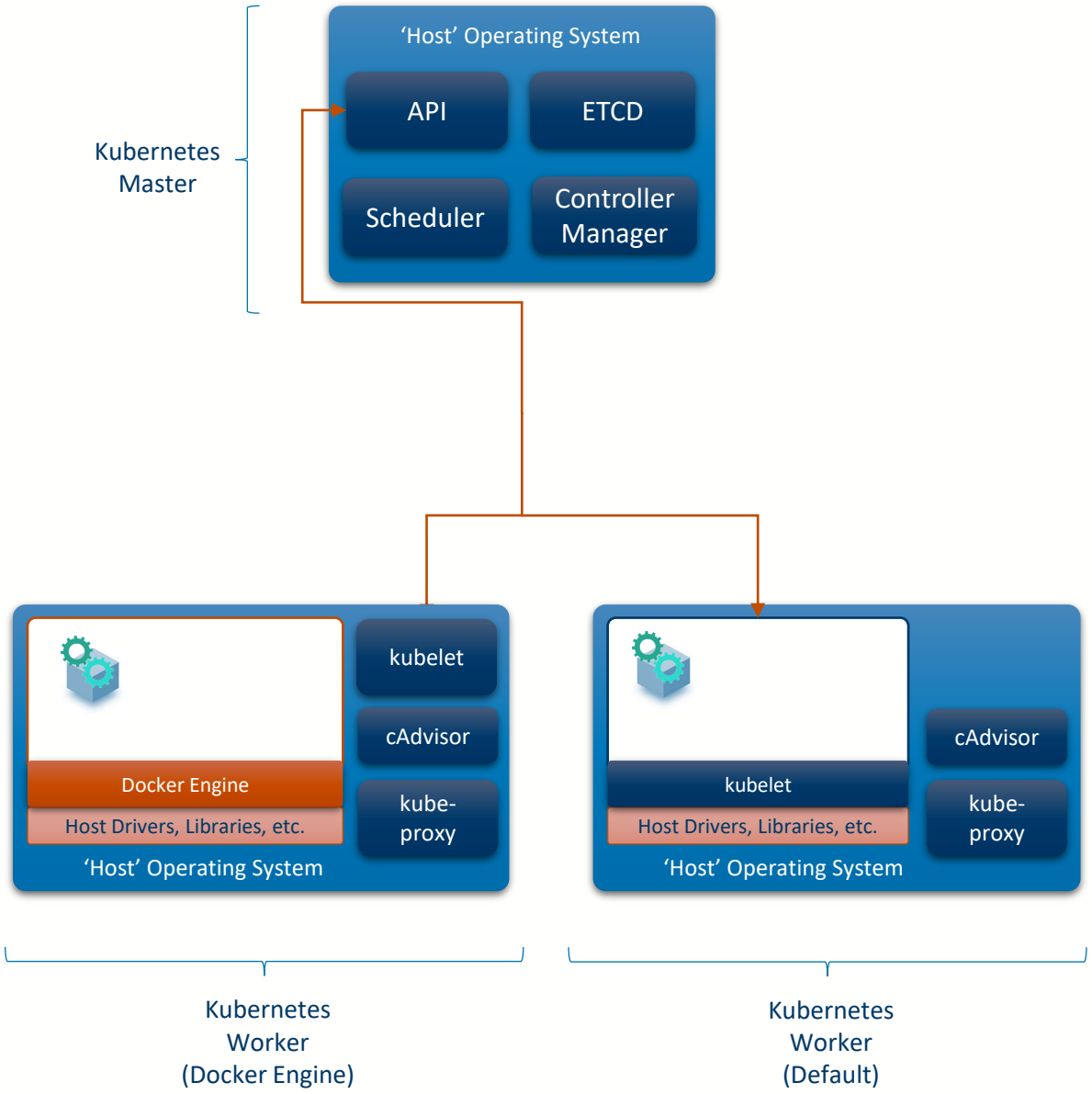
Kubernetes

- It is useful to consider Kubernetes as being comprised of two parts
  - Kubernetes Master: Contains the API, ETCD, Scheduler and controller manager – there always needs to be 1 or more of these
  - Kubernetes Worker: An implementation of a container engine and node control services
    - Earlier versions puppeted another container engine (like Docker Engine)
    - Current versions use Kubernetes's own engine



Kubernetes

- A Kubernetes Worker is comprised of three components
  - Kubelet: Responsible for making sure containers (Pods) are running
    - Usually also the Container Engine
  - Kube-proxy: maintains network rules on nodes that allow network communication inside or outside of the cluster
  - cAdvisor: Analyzes and exposes resource usage and performance data from running containers



University of Glasgow

Kubernetes as a State Management System

- A notable difference between normal management systems and Kubernetes is how users interact with it
  - Users define a desired state, described by a Kubernetes Object
  - The Kubernetes Scheduler and Controller Manager is responsible for making sure that state is achieved (as soon as possible)

- This difference is subtle, but important:
  - Once a desired state is defined, Kubernetes will continuously try to achieve that state
    - (i.e. it continually checks that everything the user wanted is in the correct state, and will attempt recovery if not)
  - The user does not need to know how that will be accomplished

University of Glasgow

Kubernetes Objects

- **Pods, Services and Routes** are persistent entities in Kubernetes that are its core building blocks. They represent the **desired state of the cluster**:
  - What containerized applications are running (and on which nodes)
  - The resources available to those applications
  - The policies around how those applications behave, such as restart policies, upgrades, and fault-tolerance

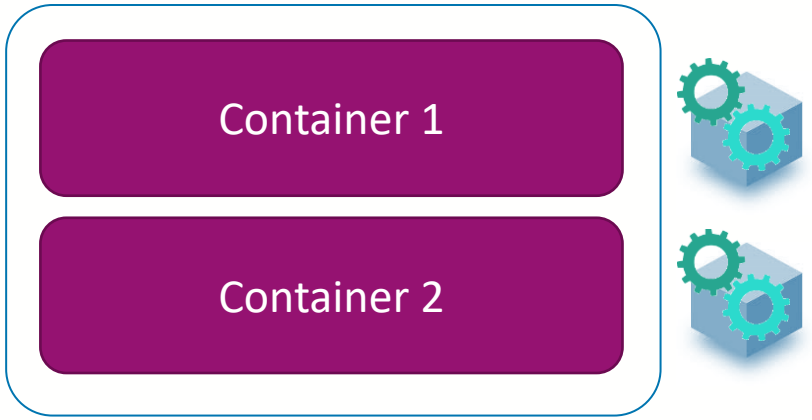
- Each of these objects is a '**record of intent**'
  - Kubernetes will constantly work to ensure that the cluster state matches that described by its objects

- We **control the cluster by creating and deleting these objects**
  - (rather than directly running commands)

University of Glasgow

Kubernetes Building Blocks: Pods

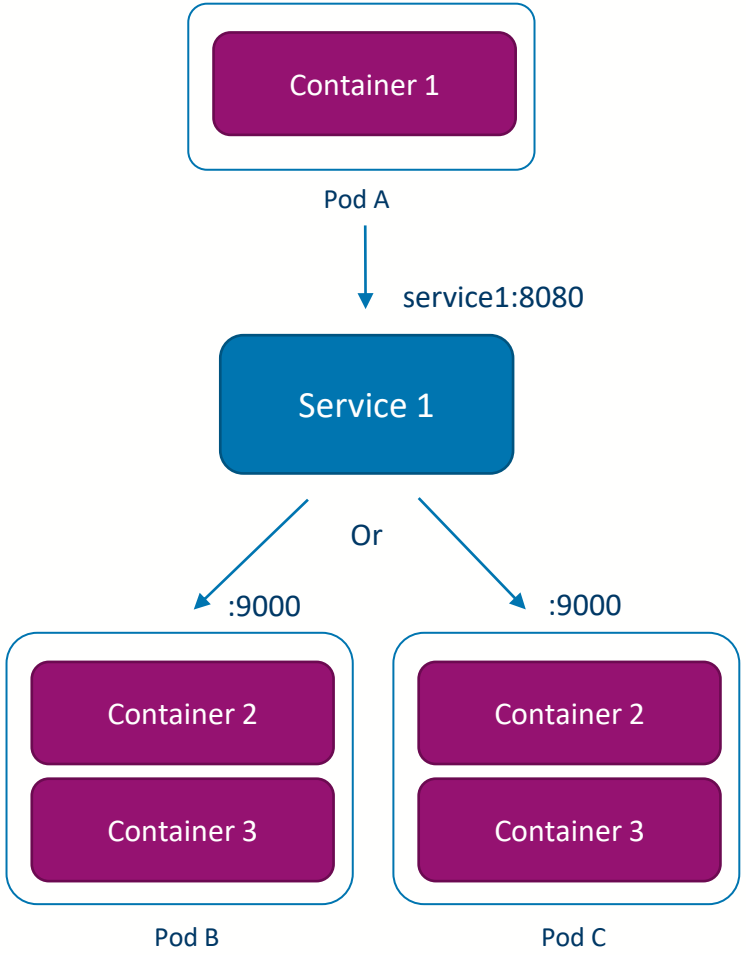
- Kubernetes does not define a container object
- Instead, it defines a **Pod** which is a **grouping** of **one or more containers**
  - To run a single container, you create a pod describing that one container
  - Pod management is the responsibility of kubelet
- Pods are the smallest ‘compute unit’ within Kubernetes, meaning that containers in a pod:
  - Share resources
  - Will always be hosted on the same machine
  - Are always visible to each other over the network
  - Are created and destroyed together (if one dies the others are killed)



Pod 1

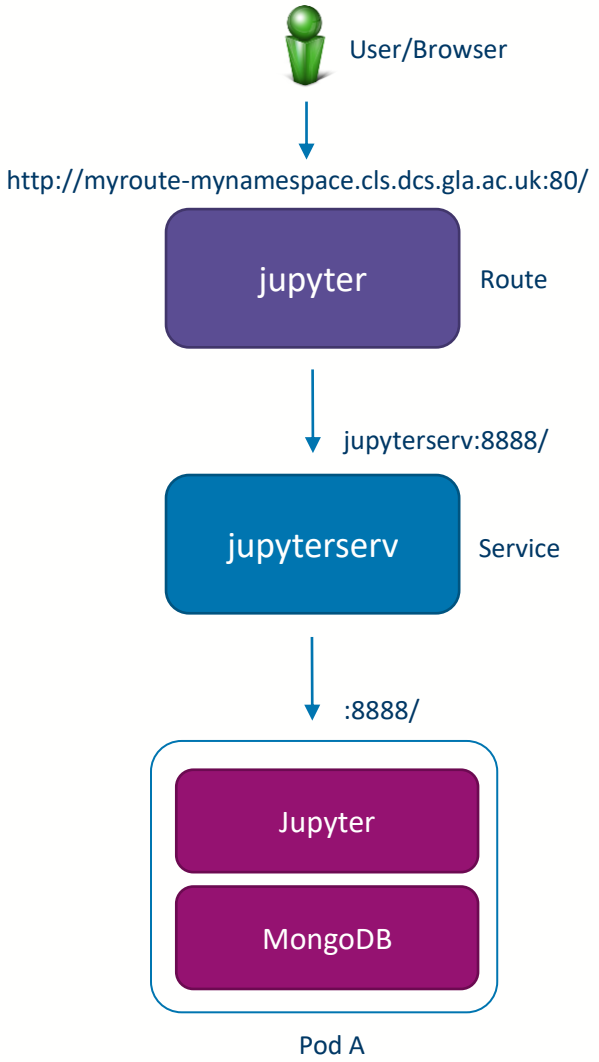
Kubernetes Building Blocks: Services

- When a Pod is launched, we won't know (and often don't care) on what physical machine that Pod is running
  - Indeed, that location may change over time based on machine availability
- However, if we are building applications that need to communicate with one or more Pods then this is a major issue
  - To what IP address do I use to direct traffic for my Pods?
- This is handled via objects called **Services**
  - Services are **visible to all Pods** across the network (within your project)
  - Services have an **internal hostname** via which they can be reached
  - **Traffic** directed at a service will be **re-routed** to designated Pod(s) based on rules and **may change ports**
  - Can also act as a **load-balancer** for multiple Pods
  - Services are managed by kube-proxy



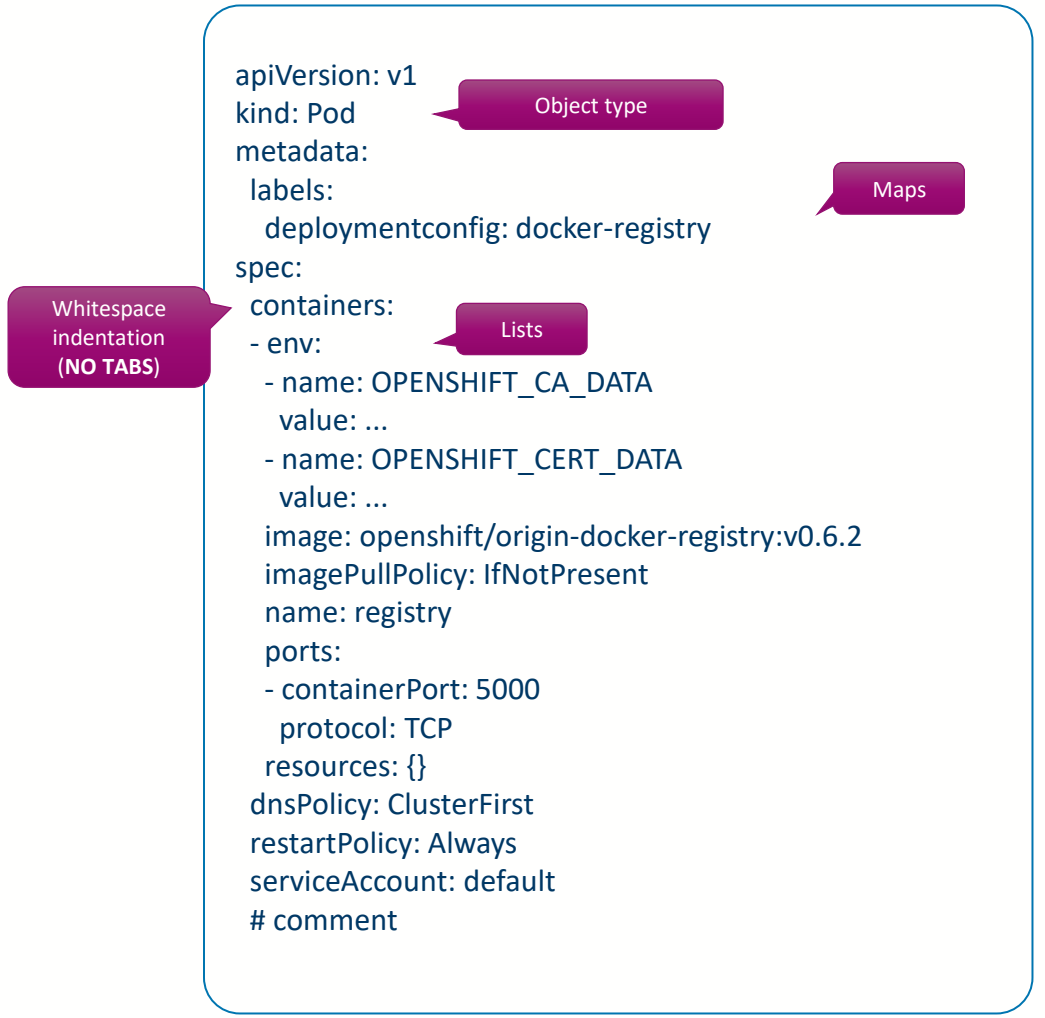
Kubernetes Building Blocks: Routes

- However, services don't expose network services outside a cluster, i.e. are not externally visible
- Instead, we also need a **Route object** to make services visible outside the cluster
  - A **Route** exposes a service at a **defined hostname** for external access
- For instance in our local cluster, Routes are exposed via
  - **http://**<routename>-<namespace>.<clusterid>**.dcs.gla.ac.uk:80/**



Defining objects in YAML

- Typically, to create new objects, we write a plain text file in YAML format
  - "YAML Ain't Markup Language“
  - It’s a human-readable format that targets many of the same communications applications as XML but has a minimal syntax



Hello World

- A basic Pod that simply prints ‘hello world’ can be launched using this pod description
  - **kind: Pod** – This describes a Kubernetes Pod object
  - **metadata > name: helloworld** – The name of the Pod
  - **metadata > namespace: bla** – The namespace to start the Pod within
  - **spec > containers** – The list of containers in the pod
    - **name** – name of the container
    - **image** – the container image (defaults to searching dockerhub)
    - **imagePullPolicy** – do we always download a new image from the repository?
    - **command** – the container command to run, ‘echo’ in this case
    - **args** – the command arguments, i.e. “hello world”
  - **spec > restartPolicy** – if the pod exits do we restart it?

```
apiVersion: v1
kind: Pod
metadata:
  name: helloworld
  namespace: bla
spec:
  containers:
  - name: hwcontainer
    image: rsippl/centos-dev:latest
    imagePullPolicy: IfNotPresent
    command:
      - 'echo'
    args:
      - 'hello world'
  restartPolicy: Never
```

University of Glasgow

Advantages and Disadvantages of Kubernetes

Advantages

- Is the defacto standard (nearly everyone uses it)
- Is reliable and lightweight
- Resource allocation to containers is granular
- Can perform network load-balancing
- Supports Docker Containers

Disadvantages

- Not the easiest to get started with
  - We just scratched the surface with objects, there are many more
  - Storage management can be tricky
- Debugging can be difficult when something goes wrong

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers
@UofGlasgow